



Katedra informatiky a výpočetní techniky

Semestrální práce z předmětu
Paralelní programování

Simulace měření kvantového stavu

Autor:

Bc. Stanislav Kafara

A24N0088P

skafara@students.zcu.cz

Obsah

1	Zadání	2
2	Teoretické pozadí	3
2.1	Kvantový výpočet	3
2.2	Simulace měření	3
3	Návrh řešení	5
3.1	Vzorkování z pravděpodobnostní distribuce	5
3.2	Generování náhodných čísel	6
4	Popis implementace	7
5	Výsledky	8
5.1	Robustnost Philox PRNG	8
5.2	Výpočetní čas	9
5.3	Poznámky	9
6	Uživatelská příručka	11
6.1	Překlad a sestavení	11
6.2	Spuštění	12
7	Závěr	13
A	Diagram řešení	15
B	Statistická podobnost distribucí naměřených výsledků	16
C	Urychlení výpočetního času	19

Kapitola 1

Zadání

Cílem semestrální práce je navrhnout a implementovat simulaci měření stavu kvantového výpočtu. Výstupem bude program se třemi způsoby výpočtu: sekvenční skalární, vícevláknový s manuální vektorizací (**AVX2**) a paralelní s využitím GPU. Řešení bude realizováno jako dynamicky linkovaná knihovna (DLL) k existující implementaci simulátoru kvantového počítače. Výsledná implementace bude otestována na správnost, bude porovnána rychlost výpočtu a robustnost výsledků v porovnání se skutečně náhodnou simulací.

Kapitola 2

Teoretické pozadí

2.1 Kvantový výpočet

Kvantový výpočet představuje výpočetní paradigma založené na principech kvantové mechaniky, které se v zásadních aspektech liší od klasického pojetí výpočtu. Zatímco klasické počítače pracují s bity nabývajících hodnot 0 nebo 1, kvantové počítače využívají kvantové bity (qubity), jejichž stav může být popsán jako lineární kombinace obou těchto hodnot. Tato vlastnost, označovaná jako superpozice, umožňuje kvantovým systémům reprezentovat více stavů současně.

Výpočet v kvantovém počítači je realizován postupnou aplikací kvantových hradel, která odpovídají unitárním transformacím kvantového stavu. Výsledkem kvantového výpočtu není přímo konkrétní hodnota, ale kvantový stav, ze kterého je teprve pomocí měření získána klasická informace. Vzhledem k tomu, že kvantové systémy podléhají pravděpodobnostnímu chování, je výsledek měření obecně nedeterministický. Praktické využití kvantových algoritmů proto často vyžaduje opakované provádění výpočtu a statistické vyhodnocení naměřených výsledků.

2.2 Simulace měření

Měření kvantového stavu je finální část kvantového výpočtu, při kterém je kvantová informace převedena na klasický výsledek. Kvantový stav systému je popsán stavovým vektorem, jehož jednotlivé složky odpovídají amplitudám pravděpodobností měřitelných stavů. Pravděpodobnost získání konkrétního výsledku měření je dána druhou mocninou absolutní hodnoty příslušné amplitudy.

V případě simulace se jedná o numerickou operaci nad stavovým vektorem.

rem. Simulace měření spočívá ve výpočtu pravděpodobnostního rozdělení všech možných výsledků a následném náhodném výběru jednoho z nich. Tento výběr je typicky realizován pomocí generátoru pseudonáhodných čísel, přičemž výsledná hodnota je v naivním případě určena porovnáním generovaného náhodného čísla s kumulativní distribuční funkcí odpovídající danému kvantovému stavu.

Kapitola 3

Návrh řešení

Řešení bude realizováno v programovacím jazyce C++ jako dynamicky linkovaná knihovna (DLL) k existující implementaci simulátoru kvantového počítače. Výpočty na GPU budou realizovány pomocí OpenCL. Vektorizace bude realizována manuálně pomocí intrinsics podle standardu AVX2. Paralelizace bude realizovaná pomocí funkcí standardní knihovny a knihovny oneTBB. Jednotlivé implementace způsobů výpočtu bude realizováno pomocí specializací šablonových funkcí. Řešení je vizualizováno diagramem A.1 a skládá se z šesti hlavních kroků:

1. zavolání knihovní funkce ze simulátoru,
2. předzpracování obdržených dat,
3. výpočet pravděpodobnostní distribuce,
4. předzpracování pravděpodobnostní distribuce,
5. vzorkování z pravděpodobnostní distribuce a
6. zápis vzorků do výstupního pole.

3.1 Vzorkování z pravděpodobnostní distribuce

Vzhledem k velikosti stavového prostoru kvantového výpočtu, která roste exponenciálně s počtem qubitů, je efektivní realizace tohoto kroku zásadní z hlediska výpočetní náročnosti celé simulace. V naivním přístupu je vzorkování realizováno porovnáním náhodně vygenerované hodnoty s kumulativní distribuční funkcí, což vyžaduje lineární průchod polem pravděpodobností.

Tento postup má časovou složitost $\mathcal{O}(n)$ na jeden vzorek, kde n odpovídá počtu možných měřitelných stavů.

Z tohoto důvodu je v rámci navrženého řešení použita *Voseova aliasová metoda* [1], která umožňuje vzorkování z diskrétní pravděpodobnostní distribuce v konstantním čase $\mathcal{O}(1)$ po jednorázovém předzpracování distribuce s lineární složitostí $\mathcal{O}(n)$. Metoda spočívá v transformaci původní distribuce do dvou pomocných polí, která pro každý stav uchovávají pravděpodobnostní práh a případný aliasový index.

Během samotného vzorkování je následně náhodně zvolen index a pomocí další náhodné hodnoty je rozhodnuto, zda je vybrán přímo odpovídající stav, nebo jeho alias. Tento postup je dobře paralelizovatelný a vhodný jak pro vektorové zpracování na CPU, tak pro implementaci na GPU.

3.2 Generování náhodných čísel

Proces vzorkování z pravděpodobnostní distribuce je inherentně založen na náhodném výběru. Kvalita použitého zdroje náhody má přímý vliv na statistické vlastnosti výsledků simulace. Nevhodně zvolený generátor pseudonáhodných čísel může vést ke vzniku korelací, systematických odchylek a zkreslení výsledné distribuce naměřených výsledků. Za dobrý zdroj náhody můžeme považovat např. službu *Random.org* [2], která poskytuje zdroj skutečně náhodných čísel odvozených z naměřených atmosférických dat.

V rámci navrženého řešení je použit pseudonáhodný generátor *Philox* [3], který patří do třídy tzv. *counter-based* generátorů. Na rozdíl od tradičních stavových generátorů, jako je například Mersenne Twister, neudrží *Philox* vnitřní stav závislý na předchozích generovaných hodnotách. Každá náhodná hodnota je deterministicky vypočtena jako funkce čítače a klíče.

Tato vlastnost činí generátor *Philox* zvláště vhodným pro paralelní výpočty. V prostředí GPU výpočtu nebo vícevláknového zpracování lze každému vláknu přiřadit nezávislou část prostoru čítače bez jakékoliv přídavné režie synchronizace. Zároveň je možné generovat větší bloky náhodných čísel najednou a algoritmus efektivně implementovat vektorově.

Kapitola 4

Popis implementace

Vstupním bodem je vystavená funkce `ESimulator_Run`, která přijme stavový vektor, počet měření a výstupní pole reprezentující histogram naměřených hodnot. Funkce postupně volá jednotlivé specializace šablonových funkcí podle zvoleného způsobu výpočtu.

Knihovní funkce přijme komplexní amplitudy jako `AoS` (Array of Structures) (reálná a imaginární složka). Pro další efektivní zpracování data převede na `SoA` (Structure of Arrays) odstraněním prokládání. Následně se vypočte pravděpodobnostní distribuce. Ta se dále normalizuje a pomocí Voseovy aliasové metody se vytvoří vzorkovací tabulka.

Pro naměření jednoho vzorku je potřeba vygenerovat jedno diskrétní číslo v rozsahu 0 až počet měřitelných stavů vyjma a jedno spojitě číslo v rozsahu 0 až 1 vyjma. Diskrétní číslo je počáteční výběr naměřeného stavu a spojitě číslo je prahová hodnota, do které je tento počáteční výběr akceptován, jinak je vybrán alias tohoto počátečního výběru. Pro výběr naměřeného stavu má `uint32` dostatečný rozsah a pro prahovou hodnotu byl zvolen `double` kvůli přesnosti simulace.

Philox v jednom kroku generuje čtyři 32 bitová čísla, tedy čtyři diskrétní čísla `uint32` nebo po transformaci dvě spojitá čísla `double`. Takto se předgenerují pole, které se následně použijí při hromadném vzorkování z pravděpodobnostní distribuce.

Výsledné naměřené stavy se zapíší do výstupního pole. Tato část je paralelizována tak, že se nejprve pole naměřených výsledků seřadí a provede se redukce, při které se naměřené výsledky kódují délkou běhu (RLE). Následně se výsledky zapíší do výstupního pole s předpokládaným menším počtem náhodných přístupů do paměti.

Kapitola 5

Výsledky

Veškeré měření probíhalo na stolním počítači s následujícími specifikacemi:

- OS - Windows 10
- CPU - Intel i5-7600 (4 jádra)
- RAM - 16 GB
- GPU - Nvidia GeForce GTX 1060 6 GB

5.1 Robustnost Philox PRNG

Robustnost Philox PRNG byla měřena porovnáním distribucí naměřených výsledků pomocí Philox PRNG a Random.org. Celkem bylo naměřeno až 32M výsledků nad stavovým vektorem nad 10 qubity (16 KB). Stavový vektor byl uměle vygenerován podle několika známých pravděpodobnostních distribucí.

Statistická podobnost distribucí naměřených výsledků byla měřena pomocí dvou metrik, a sice *celkovou variační vzdáleností* (TVD) a *Jensenovo-Shanonovo divergencí* (JSD). Celková variační vzdálenost vyjadřuje maximální rozdíl mezi pravděpodobnostmi přiřazenými stejným událostem ve dvou porovnávaných rozděleních. Nabývá hodnot v intervalu $[0, 1]$, přičemž hodnota 0 značí totožné distribuce a vyšší hodnoty indikují rostoucí míru odlišnosti. Jensenova-Shannonova divergence je symetrická a vždy konečná míra rozdílu mezi dvěma pravděpodobnostními rozděleními, odvozená z Kullbackovy-Leiblerovy divergence. Na rozdíl od KL divergence je JSD numericky stabilní a omezená, přičemž hodnota 0 odpovídá identickým distribucím a rostoucí hodnota značí zvyšující se rozdíl mezi nimi.

Výsledky jsou shrnuty v tabulce B.1. Jejich průběh v závislosti na počtu vzorků je vizualizován v grafu B.1 a B.2. Pokud budeme předpokládat, že dobrá shoda nastává při $TVD < 0.05$ nebo $JSD < 0.01$, pak je potřeba alespoň 64-128K měření, aby se distribuce výsledků přiblížila té naměřené pomocí skutečné náhody.

5.2 Výpočetní čas

Výpočetní čas byl měřen jako průměr opakovaných měření jednotlivých operací. V případě výpočtu na GPU se jedná o výpočetní čas samotného kernelu. Časy výpočtu a jejich urychlení je shrnuto v tabulce C.1 a C.2. Jejich průběh je vizualizován v grafech C.1, C.2, C.3 a C.4.

Výpočet pravděpodobností dosáhl urychlení 1.40, respektive 7.76 a paralelizace urychlila výpočet za předpokladu, že se počítá alespoň nad stavovým vektorem o 32K stavech. Generování pseudonáhodných diskretních čísel dosáhlo urychlení 3.99, respektive 77.58 a paralelizace urychlila výpočet za předpokladu, že se generuje alespoň 4K čísel. Generování pseudonáhodných spojitých čísel dosáhlo urychlení 3.89, respektive 137.80 a paralelizace urychlila výpočet za předpokladu, že se generuje alespoň 2K čísel. Vzorkování z pravděpodobnostní distribuce dosáhlo urychlení 2.05, respektive 14.11 a paralelizace urychlila výpočet za předpokladu, že se vzorkuje alespoň 8K čísel. Zápis naměřených výsledků do výstupního pole dosáhl urychlení 3.39 a paralelizace urychlila výpočet za předpokladu, že bylo naměřeno alespoň 1M výsledků.

Výpočetní čas celého procesu simulace měření je 14 599 ms v případě sekvencního skalárního výpočtu, 10 837 ms v případě vícevláknového výpočtu s vektorizací a 8 300 ms v případě paralelního výpočtu s využitím GPU. Celkové urychlení vícevláknovým výpočtem s vektorizací dosahuje $S_{par} = 1.35$ a celkový výpočetní čas se zkrátil o 25.77 %. Karp-Flattova metrika nabývá hodnoty $e_{par} = 0.73$, za předpokladu $p = 16$, tj. 4 jádra a 4 linky SIMD vektoru pro převážně používaný datový typ double. V případě paralelního výpočtu s využitím GPU dosahuje celkové zrychlení $S_{gpu} = 1.76$ a celkový výpočetní čas se zkrátil o 43.14 %. Karp-Flattova metrika nabývá hodnoty $e_{gpu} = 0.57$, za předpokladu $p = 1280$ CUDA jader.

5.3 Poznámky

Nízké urychlení několika částí výpočtu i vysoká Karp-Flattova metrika celého procesu měření může být vysvětlena primárně tím, že nezanedbatelná

část výpočtu je velmi málo aritmeticky náročná a paměťově vázaná, jako např. odstranění prokládání, výpočet pravděpodobnosti nebo vzorkování z pravděpodobnostní distribuce. Dále, samotné sestavení vzorkovací tabulky je převážně neparalelizovatelné a časově netriviálně náročné. V případě paralelního výpočtu na GPU se navíc započítává režie přesunu dat na GPU a zpět.

Nezanedbatelného zlepšení jsem dosáhl, když jsem při výpočtu (zápisu) pravděpodobností využil instrukci `_mm256_stream_pd` místo `_mm256_store_pd`. Při použití této instrukce se obchází cache procesoru a nedochází ke *cache polluting*.

Kapitola 6

Uživatelská příručka

Řešení vyžaduje překladač MSCV jazyka C++ podporující standard C++20, překladač FPC jazyka Pascal (Object Pascal), GPU a potřebné ovladače OpenCL.

6.1 Překlad a sestavení

Po zadání příkazu:

```
$ Setup.bat
```

dávkový soubor nastaví prostředí pro překlad aplikace a stáhne potřebné knihovny pro běh aplikace (OpenCL, oneTBB).

Po zadání příkazu:

```
$ Build.bat [<ExecutionPolicy> [<PrngAlgorithm>]]
```

dávkový soubor přeloží simulátor kvantového počítače, knihovnu pro simulaci měření stavu kvantového výpočtu, připojí závislosti a sestaví celkovou aplikaci s demonstračním kvantovým obvodem. Výsledný spustitelný soubor, dynamicky linkovaná knihovna a další závislosti se nachází v adresáři `bin`.

ExecutionPolicy

- Sequential - sekvenční skalární
- Parallel - vícevláknový s manuální vektorizací
- Accelerated - paralelní s využitím GPU

PrngAlgorithm

- Philox - vlastní implementace Philox

- MT19937 - knihovní implementace Mersenne Twister
- RandomOrg - archiv skutečně náhodných čísel

6.2 Spuštění

Demonstrační aplikace se spustí po zadání příkazu:

```
$ Run.bat
```

Kapitola 7

Závěr

Cílem této práce bylo navrhnout a implementovat simulaci měření kvantového stavu s důrazem na efektivní paralelní zpracování. Byla realizována knihovní implementace ve třech variantách: sekvenční skalární, vícevláknová s manuální vektorizací pomocí instrukcí AVX2 a paralelní s využitím GPU prostřednictvím OpenCL. Řešení bylo integrováno jako dynamicky linkovaná knihovna k existujícímu simulátoru kvantového počítače.

V rámci práce byl navržen a implementován efektivní proces simulace měření. Statistická kvalita generovaných náhodných hodnot byla ověřena porovnáním s daty ze služby Random.org. Výsledky ukázaly, že při dostatečném počtu měření dosahuje Philox PRNG velmi dobré shody se skutečně náhodným zdrojem, přičemž metriky celkové variační vzdálenosti a Jensenovy–Shannonovy divergence s rostoucím počtem vzorků konvergují k nízkým hodnotám.

Měření ověřila, že paralelizace výpočtu vedou k výraznému zkrácení výpočetního času, zejména u generování pseudonáhodných čísel. Zároveň se ukázalo, že některé části výpočtu jsou výrazně paměťově vázané nebo obtížně paralelizovatelné, což omezuje dosažitelné urychlení.

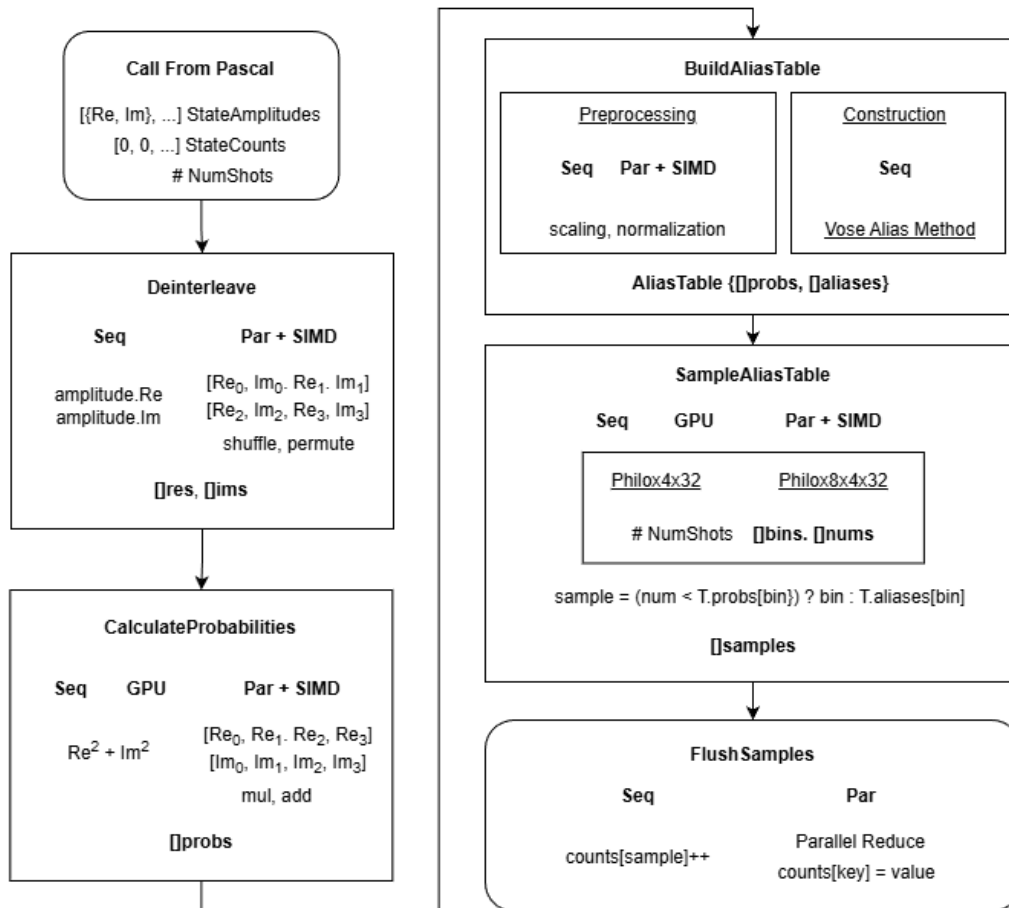
Navržené řešení splňuje stanovené cíle práce.

Bibliografie

- [1] M. Vose, “A linear algorithm for generating random numbers with a given distribution,” *IEEE Transactions on Software Engineering*, roč. 17, č. 9, s. 972–975, 1991.
- [2] M. Haahr, *RANDOM.ORG: True Random Number Service*, <https://www.random.org/>, Accessed: 2026-01-08, 2026.
- [3] J. K. Salmon, M. A. Moraes, R. O. Dror a D. E. Shaw, “Parallel Random Numbers: As Easy as 1, 2, 3,” in *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC11)*, Seattle, Washington, USA: ACM, 2011, s. 1–12.

Příloha A

Diagram řešení



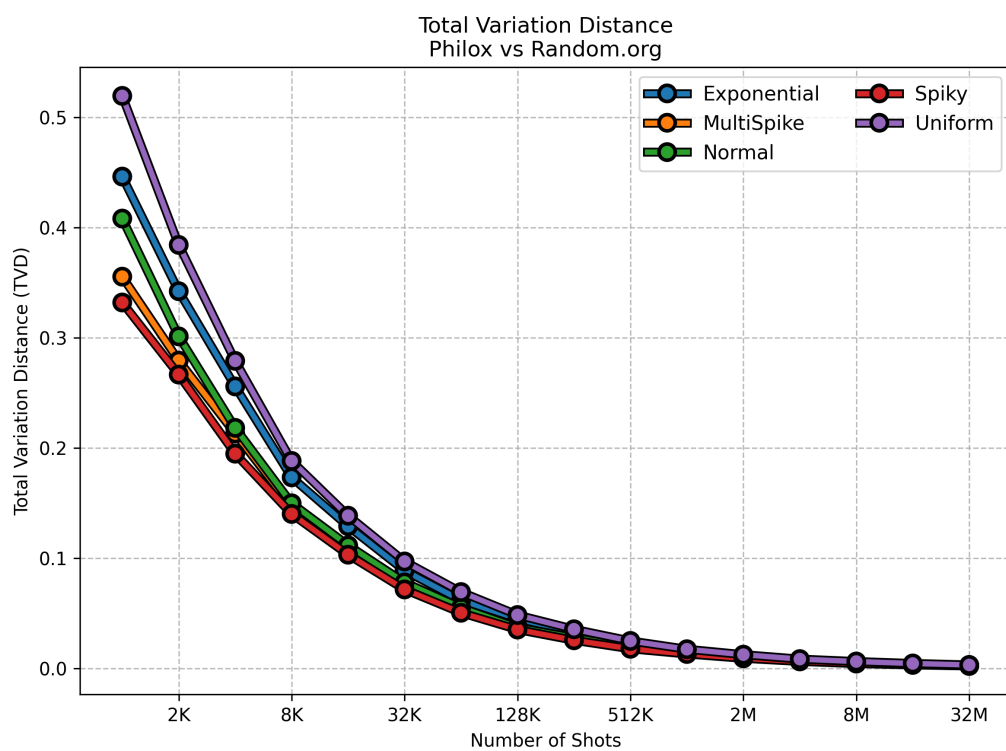
Obrázek A.1: Diagram řešení

Příloha B

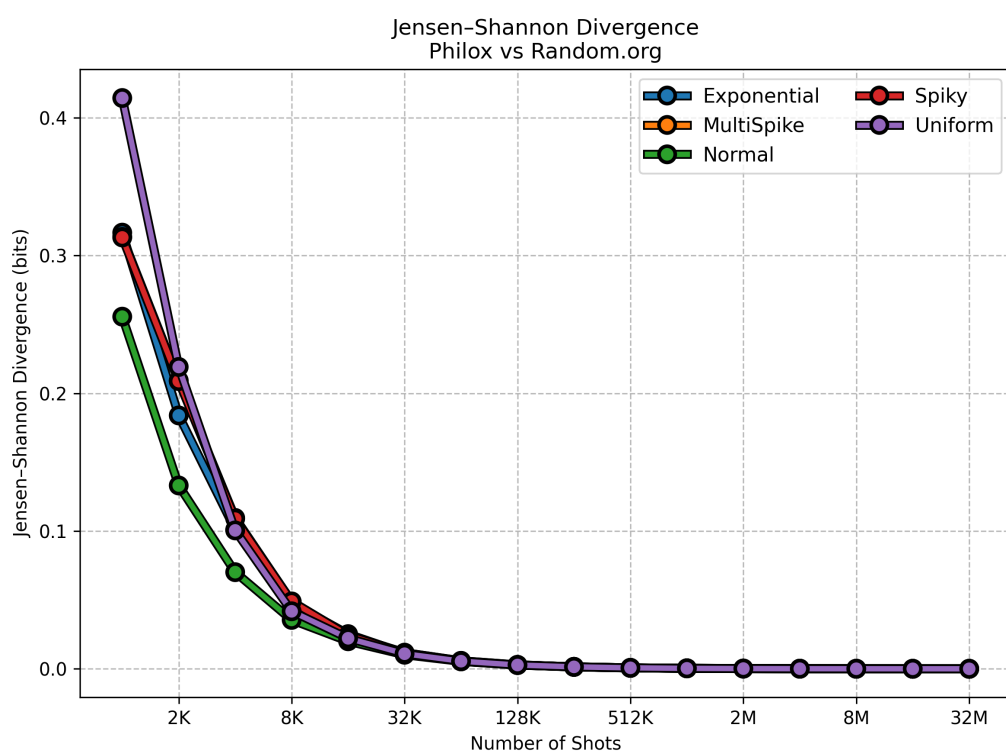
Statistická podobnost distribucí naměřených výsledků

Tabulka B.1: Statistická podobnost distribucí naměřených výsledků

Shots	Distribuce									
	Uniform		Normal		Spiky		MultiSpike		Exponential	
	TVD	JSD	TVD	JSD	TVD	JSD	TVD	JSD	TVD	JSD
1K	0.520	0.414	0.408	0.256	0.332	0.313	0.355	0.314	0.446	0.317
2K	0.384	0.219	0.301	0.133	0.267	0.209	0.279	0.209	0.342	0.184
4K	0.279	0.100	0.218	0.070	0.195	0.109	0.213	0.110	0.256	0.101
8K	0.188	0.042	0.150	0.035	0.140	0.049	0.147	0.049	0.173	0.048
16K	0.139	0.022	0.112	0.020	0.103	0.024	0.111	0.024	0.129	0.025
32K	0.097	0.011	0.078	0.010	0.072	0.012	0.077	0.012	0.089	0.011
64K	0.069	0.006	0.055	0.005	0.050	0.006	0.053	0.006	0.062	0.005
128K	0.048	0.003	0.037	0.003	0.035	0.003	0.038	0.003	0.045	0.003
256K	0.035	0.001	0.028	0.001	0.026	0.001	0.027	0.001	0.032	0.001
512K	0.025	0.001	0.019	0.001	0.018	0.001	0.019	0.001	0.022	0.001
1M	0.017	0.000	0.014	0.000	0.013	0.000	0.014	0.000	0.016	0.000
2M	0.012	0.000	0.010	0.000	0.009	0.000	0.010	0.000	0.011	0.000
4M	0.008	0.000	0.007	0.000	0.006	0.000	0.007	0.000	0.008	0.000
8M	0.006	0.000	0.005	0.000	0.004	0.000	0.005	0.000	0.006	0.000
16M	0.004	0.000	0.003	0.000	0.003	0.000	0.003	0.000	0.004	0.000
32M	0.003	0.000	0.002	0.000	0.002	0.000	0.002	0.000	0.003	0.000



Obrázek B.1: Celková variační vzdálenost



Obrázek B.2: Jensenova-Shannonova divergence

Příloha C

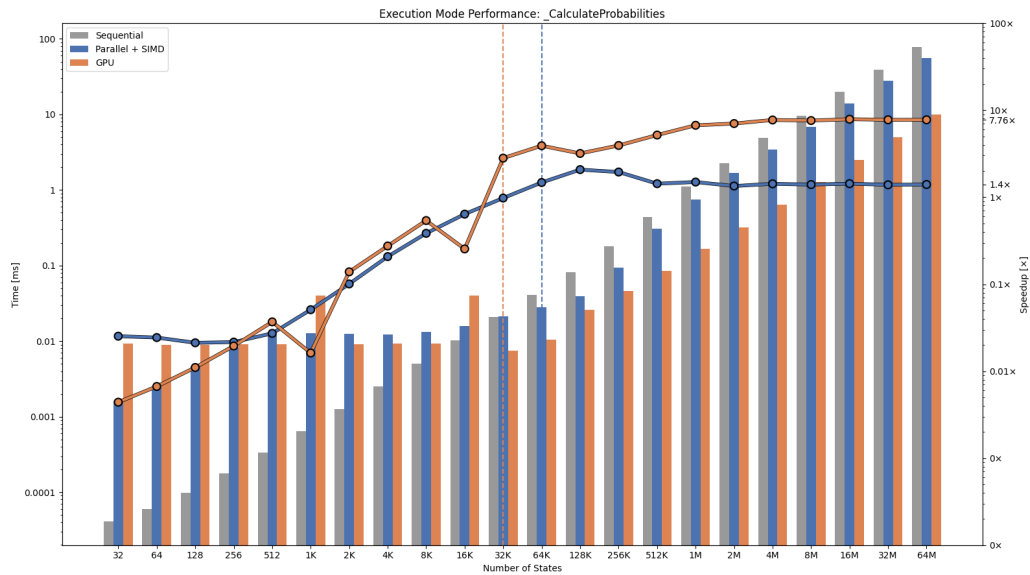
Urychlení výpočetního času

Tabulka C.1: Urychlení výpočetního času

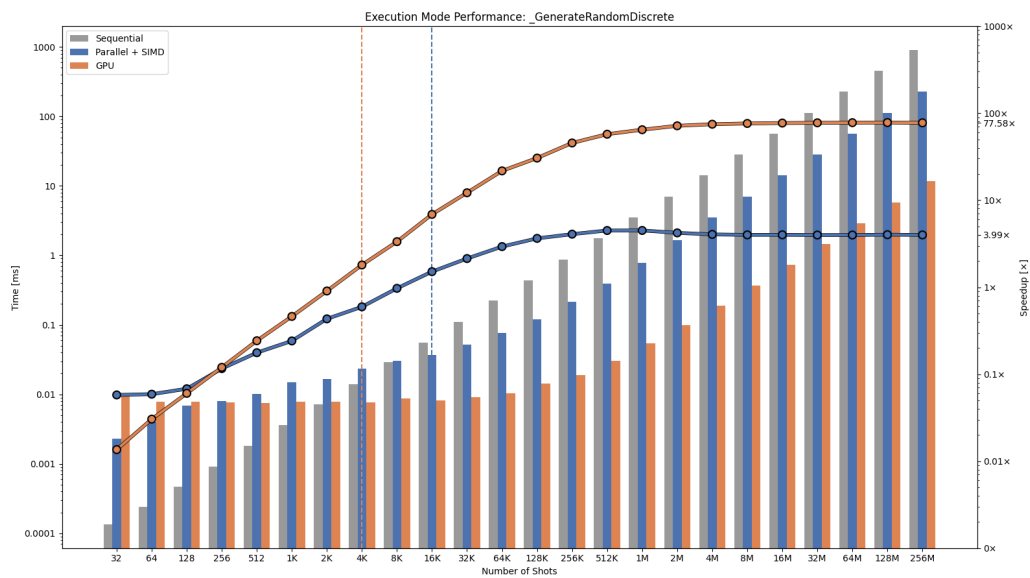
	Funkce	
States	Calc.Prob.	
	Par	GPU
32	0.03	0.00
64	0.02	0.01
128	0.02	0.01
256	0.02	0.02
512	0.03	0.04
1K	0.05	0.02
2K	0.10	0.14
4K	0.21	0.28
8K	0.38	0.54
16K	0.64	0.25
32K	0.98	2.81
64K	1.48	3.91
128K	2.08	3.19
256K	1.95	3.93
512K	1.43	5.18
1M	1.49	6.71
2M	1.35	7.03
4M	1.43	7.72
8M	1.40	7.61
16M	1.43	7.88
32M	1.39	7.76
64M	1.40	7.76

Tabulka C.2: Urychlení výpočetního času

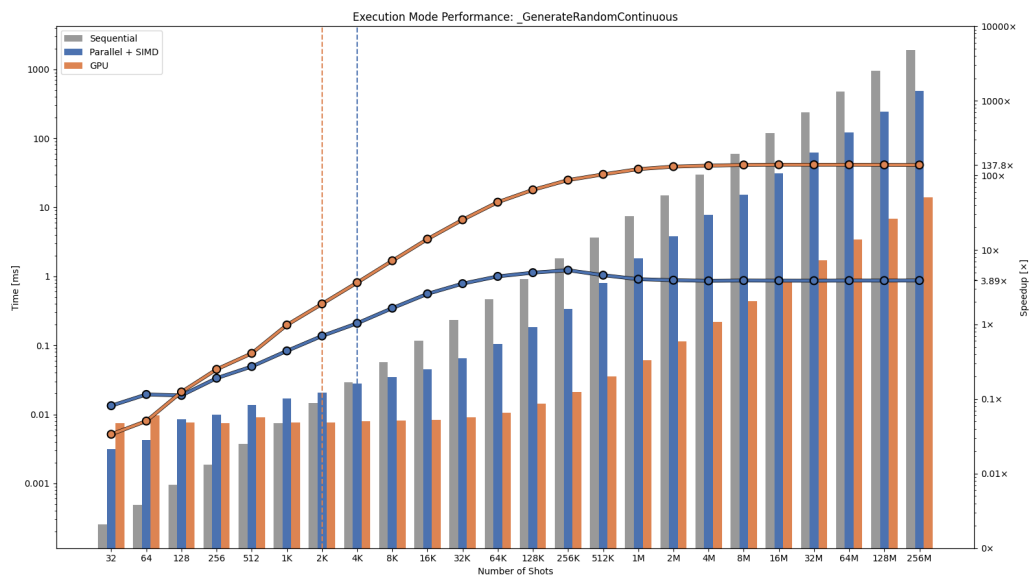
	Funkce						
Shots	Gen.R.Disc.		Gen.R.Cont.		Samp.A.Tab.		FlushSamp.
	Par	GPU	Par	GPU	Par	GPU	Par
32	0.06	0.01	0.08	0.03	0.04	0.01	0.01
64	0.06	0.03	0.12	0.05	0.03	0.01	0.01
128	0.07	0.06	0.11	0.12	0.04	0.02	0.02
256	0.12	0.12	0.19	0.25	0.05	0.03	0.07
512	0.18	0.24	0.27	0.41	0.06	0.06	0.07
1K	0.24	0.46	0.44	0.99	0.11	0.13	0.13
2K	0.43	0.91	0.70	1.89	0.15	0.26	0.34
4K	0.60	1.80	1.04	3.64	0.32	0.57	0.58
8K	0.97	3.34	1.66	7.16	0.54	1.20	0.73
16K	1.51	6.87	2.58	13.97	0.88	2.48	0.79
32K	2.13	12.18	3.53	25.36	1.41	4.36	0.78
64K	2.93	21.75	4.42	43.68	1.67	7.79	0.35
128K	3.64	30.50	4.95	63.88	1.93	7.96	0.53
256K	4.07	45.52	5.35	86.39	2.10	9.60	0.54
512K	4.49	57.38	4.58	103.10	2.28	11.81	0.72
1M	4.49	64.62	4.05	121.12	2.24	12.89	1.04
2M	4.21	71.70	3.93	131.05	2.15	13.54	1.58
4M	4.05	74.74	3.85	134.94	2.11	13.86	2.00
8M	4.00	76.28	3.89	137.98	2.08	13.95	2.49
16M	4.00	77.11	3.87	138.92	2.06	13.99	2.93
32M	3.98	77.61	3.87	138.77	2.05	14.08	3.17
64M	3.98	77.87	3.88	138.75	2.06	14.08	3.47
128M	4.02	78.07	3.88	138.29	2.05	14.12	3.37
256M	3.99	77.58	3.89	137.80	2.05	14.11	3.39



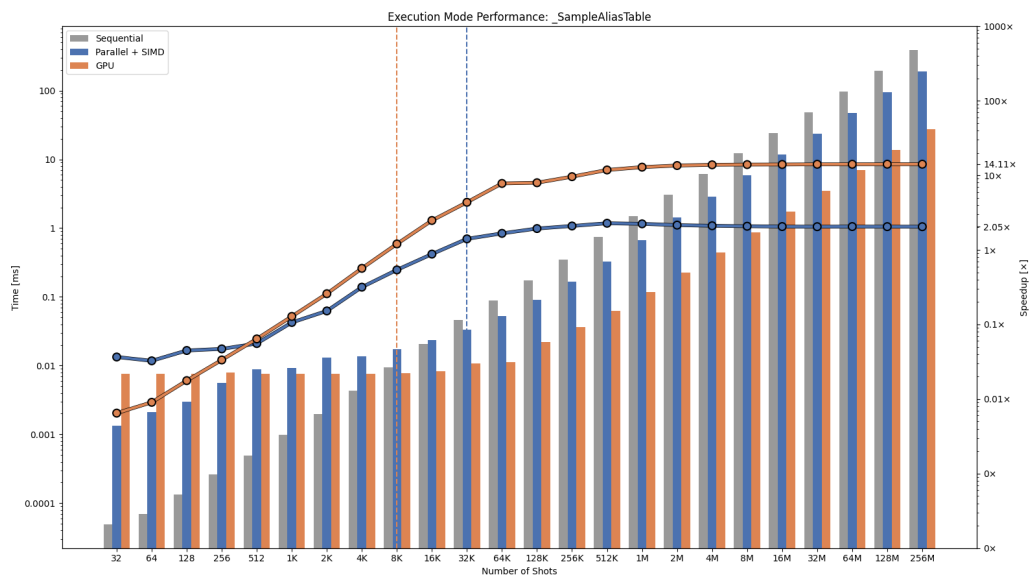
Obrázek C.1: Výpočet pravděpodobností



Obrázek C.2: Generování diskrétních pseudonáhodných čísel



Obrázek C.3: Generování spojitých pseudonáhodných čísel



Obrázek C.4: Vzorkování z pravděpodobnostní distribuce